

Problem 1: Compress Consecutive Characters

Problem Statement

You are given a string S consisting of lowercase English letters. Your task is to compress the string by replacing each group of consecutive identical characters with the character followed by its frequency.

- If a character appears only once consecutively, print only the character without the frequency.
 - The order of characters in the output must remain the same as in the original string.
-

Input Format

- A single string S .
-

Output Format

- Print the compressed string.
-

Constraints

$$1 \leq |S| \leq 10^5$$

S contains only lowercase English letters ('a' to 'z').

Example 1

Input

aaabbcccdde

Output

a3b2c4d2e

Example 2

Input

abc

Output

abc

2.Valid Parentheses

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: *s* = "()["

Output: false

Example 4:

Input: *s* = "([])"

Output: true

Example 5:

Input: *s* = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 10^4$
- *s* consists of parentheses only '()[]{}'.

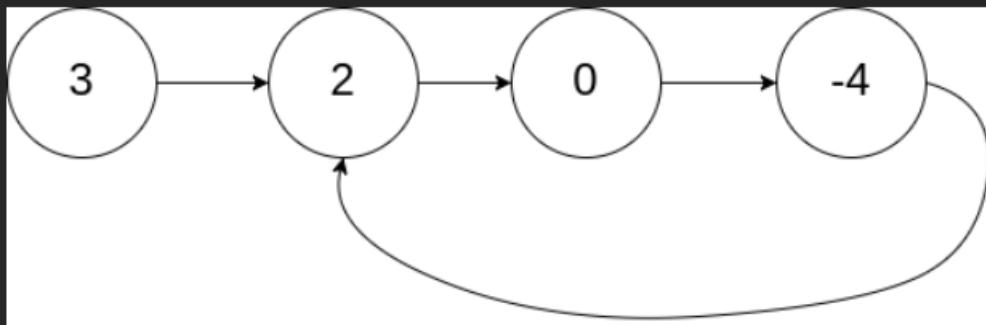
3. Linked List Cycle II

Given the head of a linked list, return *the node where the cycle begins*. If there is no cycle, return null.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to (**0-indexed**). It is -1 if there is no cycle. **Note that pos is not passed as a parameter.**

Do not modify the linked list.

Example 1:

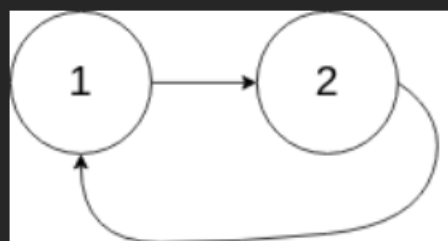


Input: head = [3,2,0,-4], pos = 1

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.

Example 2:

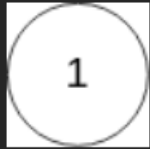


Input: head = [1,2], pos = 0

Output: tail connects to node index 0

Explanation: There is a cycle in the linked list, where tail connects to the first node.

Example 3:



Input: head = [1], pos = -1

Output: no cycle

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- pos is -1 or a **valid index** in the linked-list.

Problem 4: Sum of Minimum Values of All Subarrays

Problem Statement

You are given an integer array **arr** of size **N**.

A **subarray** is a contiguous part of the array.

For every possible subarray, determine its **minimum element** and calculate the sum of all these minimum values.

Since the answer can be very large, return it **modulo $10^9 + 7$** .

Input Format

- The first line contains an integer **N**.
- The second line contains **N** space-separated integers representing the array.

Output Format

Print a single integer representing the sum of the minimum elements of all possible subarrays.

Constraints

$$1 \leq N \leq 100000$$

$$1 \leq \text{arr}[i] \leq 1000000000$$

Example 1**Input**

4
3 1 2 4

Output

17

Explanation

Possible subarrays:

[3]
[1]
[2]
[4]
[3,1]
[1,2]
[2,4]
[3,1,2]
[1,2,4]
[3,1,2,4]

Minimum values:

3, 1, 2, 4, 1, 1, 2, 1, 1, 1

Sum = 17

Example 2**Input**

5
11 81 94 43 3

Output

444

Sample Input

3
5 4 6

Sample Output

26

Explanation

Subarrays are:

[5]
[4]
[6]
[5,4]
[4,6]
[5,4,6]

Minimum values:

$$5 + 4 + 6 + 4 + 4 + 4 = 27$$

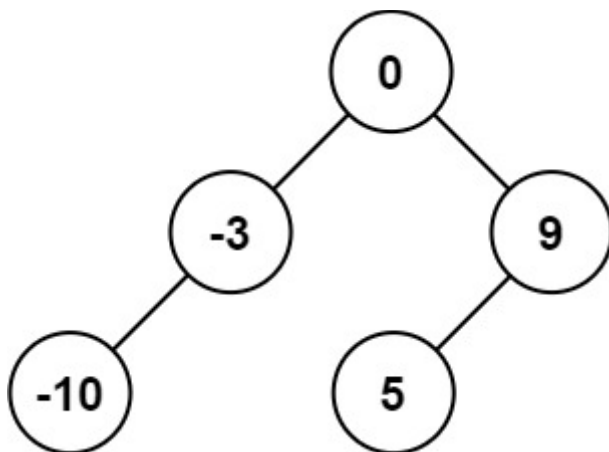
Note

- A **subarray** consists of consecutive elements of the array.
- Your solution should be efficient enough to handle the maximum constraints.
- A solution with $O(N^2)$ complexity may not pass all test cases.

5.Convert Sorted Array to Binary Search Tree

Given an integer array `nums` where the elements are sorted in **ascending order**, convert *it* to a **height-balanced** binary search tree.

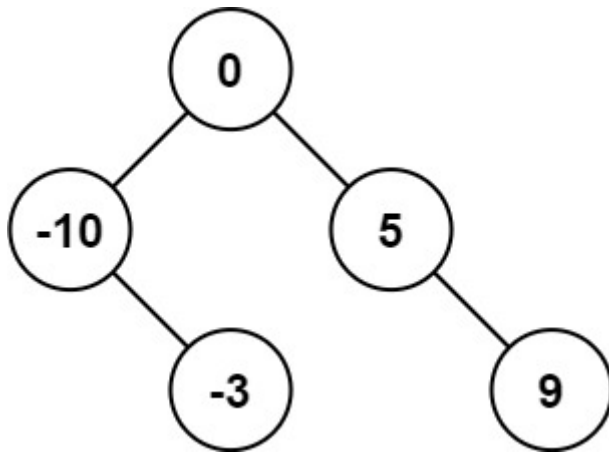
Example 1:



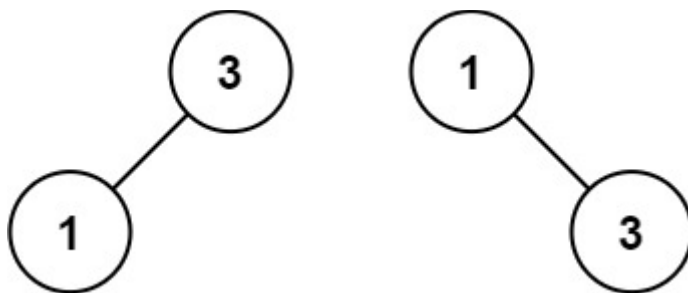
Input: `nums = [-10,-3,0,5,9]`

Output: `[0,-3,9,-10,null,5]`

Explanation: `[0,-10,5,null,-3,null,9]` is also accepted:



Example 2:



Input: nums = [1,3]

Output: [3,1]

Explanation: [1,null,3] and [3,1] are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- nums is sorted in a **strictly increasing** order.